BUSINESS INTELLIGENCE AND DATA ANALYTICS

University of Petra

Institute of Analytics
Recognized Programme

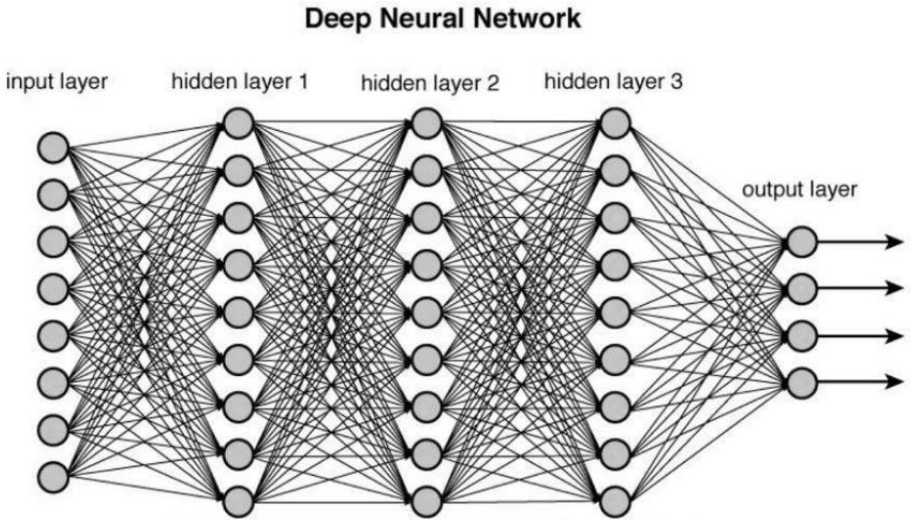
307307
Introduction to Deep Learning and Large Language Models

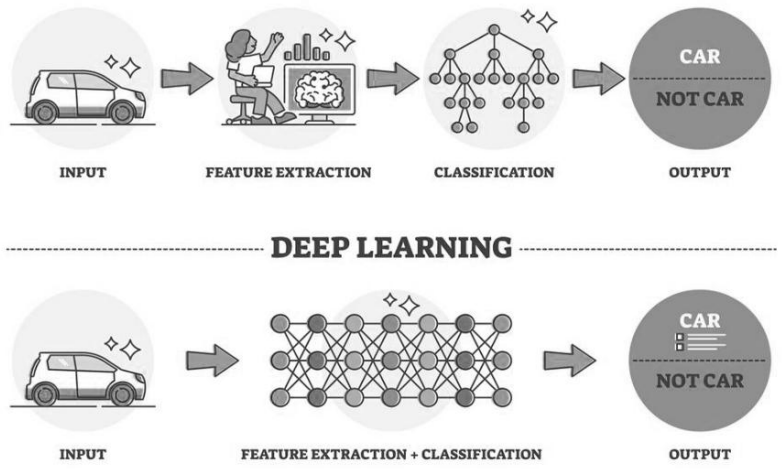
Content

- Introduction to Deep Neural Networks
  - CNNs
  - RNNs
  - The Transformer
- Contextual Word Embeddings
  - Introduction BERT
  - Introduction to HuggingFace and the Transformers Library

Introduction to Deep Learning

- Neural Networks have revolutionized artificial intelligence by enabling machines to learn from data in ways that mimic human neural processes.
- Deep neural networks (DNNs) are Neural Networks that are composed of multiple processing layers that can learn representations of data with multiple levels of abstraction.
- The power of deep learning comes from its ability to automatically discover complex patterns in raw data through the learning process, without requiring human engineers to manually specify all the knowledge needed by the computer system.





Introduction to Deep Learning

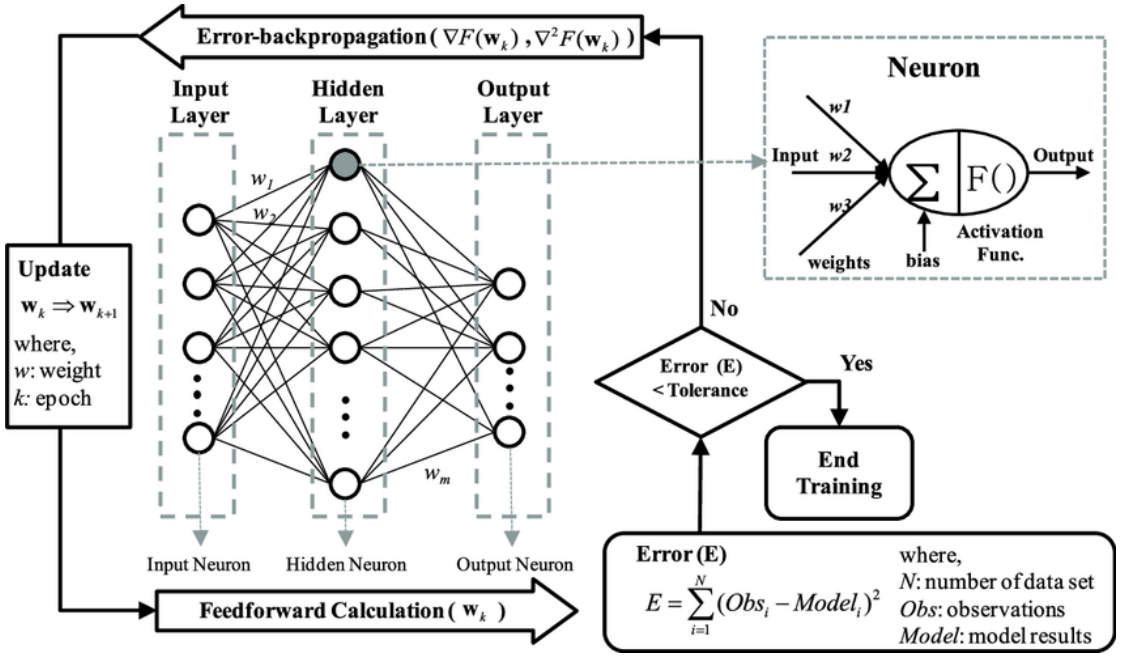
Fundamentals of Neural Networks

At their core, neural networks consist of:

1. **Neurons**: Mathematical functions that take inputs, apply weights, add a bias, and produce an output
2. **Layers**: Collections of neurons that process information in stages
3. **Activation Functions**: Non-linear functions that introduce complexity into the network
4. **Weights and Biases**: Parameters that are adjusted during training

The basic workflow involves:

- Forward propagation: Data flows through the network
- Loss calculation: The network's prediction is compared to the actual value
- Backpropagation: Errors are propagated backward to update weights
- Optimization: Weights are adjusted to minimize errors



Convolutional Neural Networks

CNNs revolutionized image processing by introducing specialized layers that mimic how the visual cortex processes information.

Key components include:

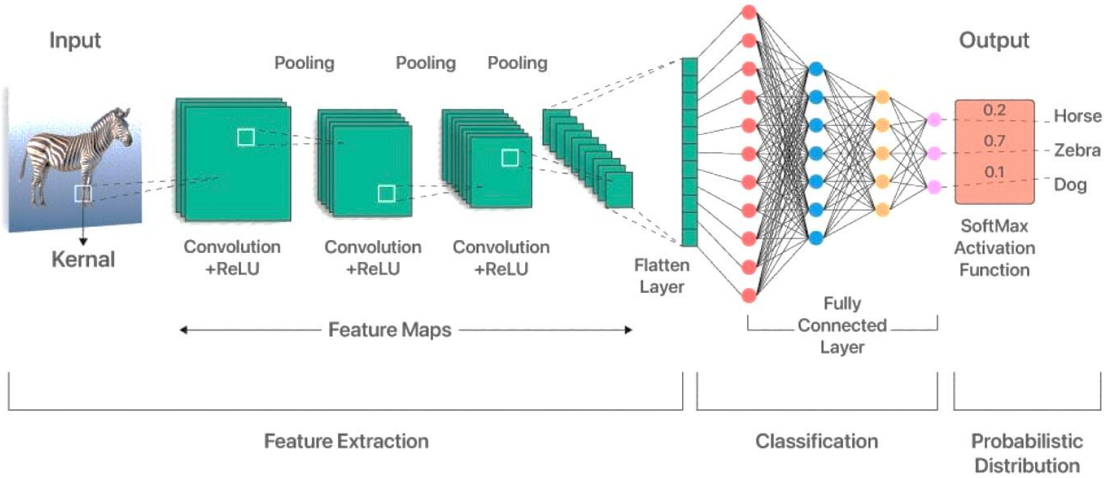
1. **Convolutional Layers**: Apply filters that scan across the input data to detect patterns
2. **Pooling Layers**: Reduce dimensions while preserving important features
3. **Fully Connected Layers**: Connect every neuron to every neuron in adjacent layers

Instead of each neuron connecting to every pixel in an image (which would be computationally expensive), CNNs use:

- **Local connectivity**: Neurons connect only to nearby pixels
- **Parameter sharing**: The same filter is applied across the entire image

Business applications include:

- Product image recognition
- Visual quality control in manufacturing
- Document processing
- Customer behavior analysis in retail



3_0	3_1	2_2	1	0
0_2	0_2	1_0	3	1
3_0	1_1	2_2	2	3
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

Recurrent Neural Networks (RNNs)

Unlike traditional neural networks, RNNs process sequences by maintaining a form of memory of previous inputs.

Key characteristics:

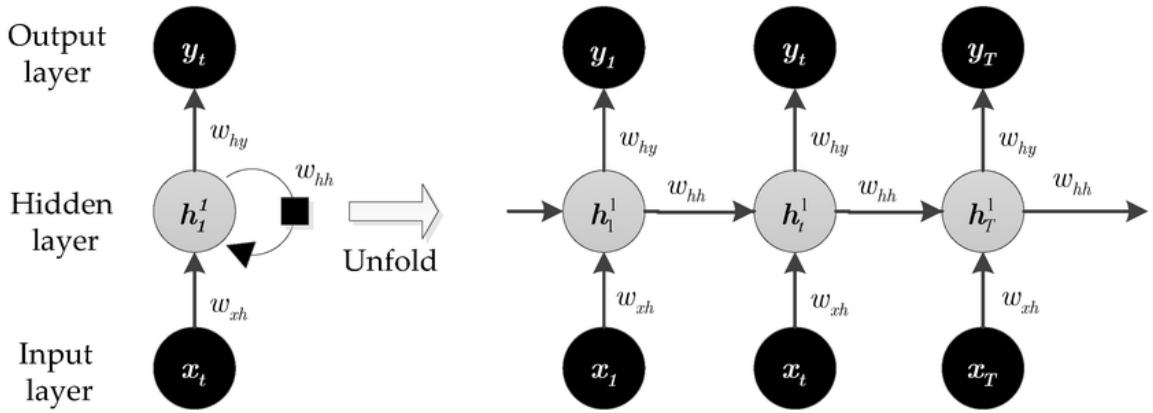
- **Time-dependent processing**: Output depends on both current and previous inputs
- **Shared parameters**: The same weights are applied at each time step
- **Memory**: Internal state acts as a form of short-term memory

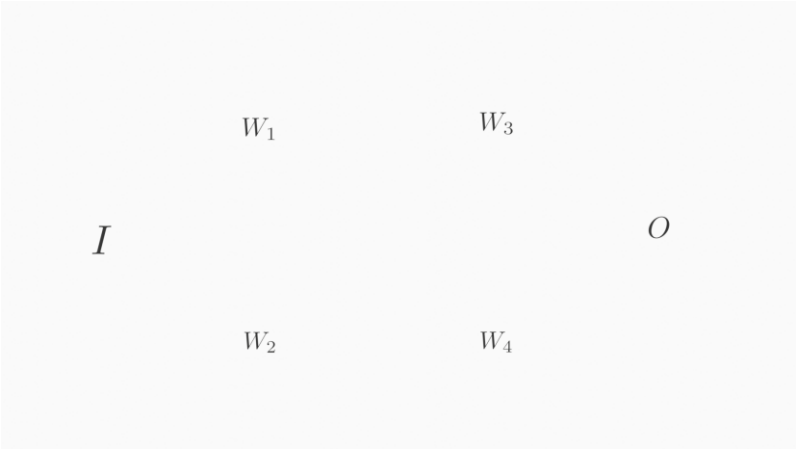
However, basic RNNs struggle with long-term dependencies due to:

- **Vanishing gradient problem**: The influence of early inputs fades over time
- **Exploding gradient problem**: Gradients grow uncontrollably during training

Business applications include:

- Language Modeling & Text Generation – Predicting the next word in a sequence (e.g., autocomplete, chatbots).
- Machine Translation – Translating text from one language to another.
- Speech Recognition – Converting spoken language into written text.
- Stock Price Prediction – Predicting future stock or financial data.
- Weather Forecasting – Modeling temporal patterns in weather data.
- Patient Monitoring – Analyzing sequences of medical data (e.g., ECG signals).
- Music Generation – Creating sequences of musical notes.
- Fraud Detection – Detecting unusual sequences in financial transactions.
- Network Intrusion Detection – Monitoring patterns of activity over time.





Transformers

Transformers – Architecture and Principles

What is a Transformer?

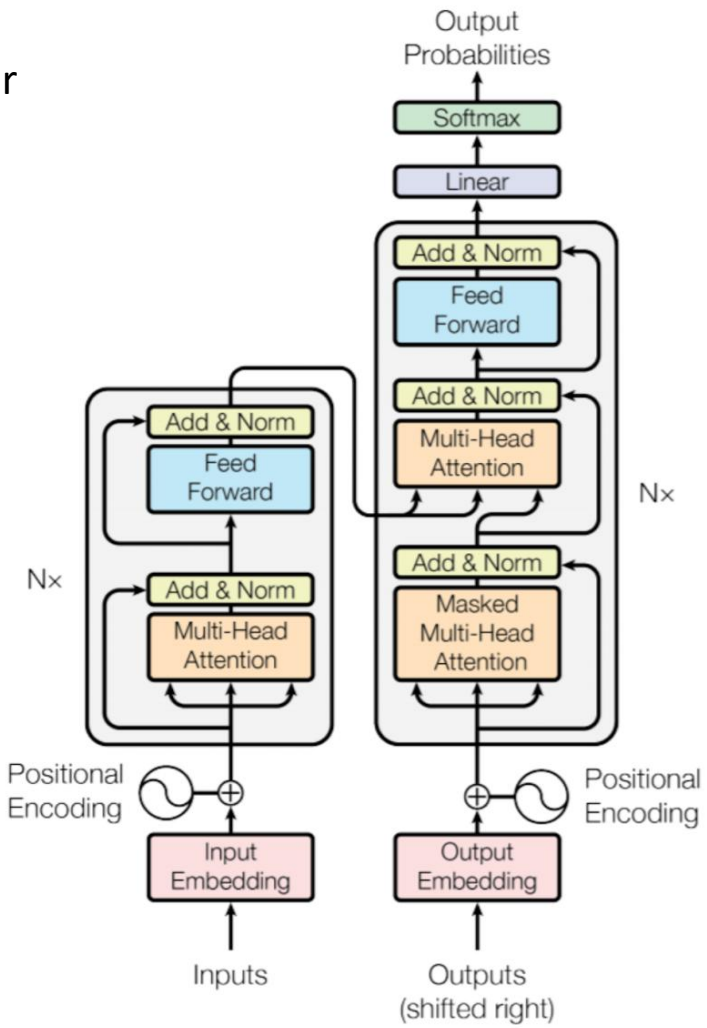
- A deep learning model based entirely on **self-attention**, with no recurrence or convolutions
- Introduced in the paper *"Attention Is All You Need"* (Vaswani et al., 2017)

Transformer Components:

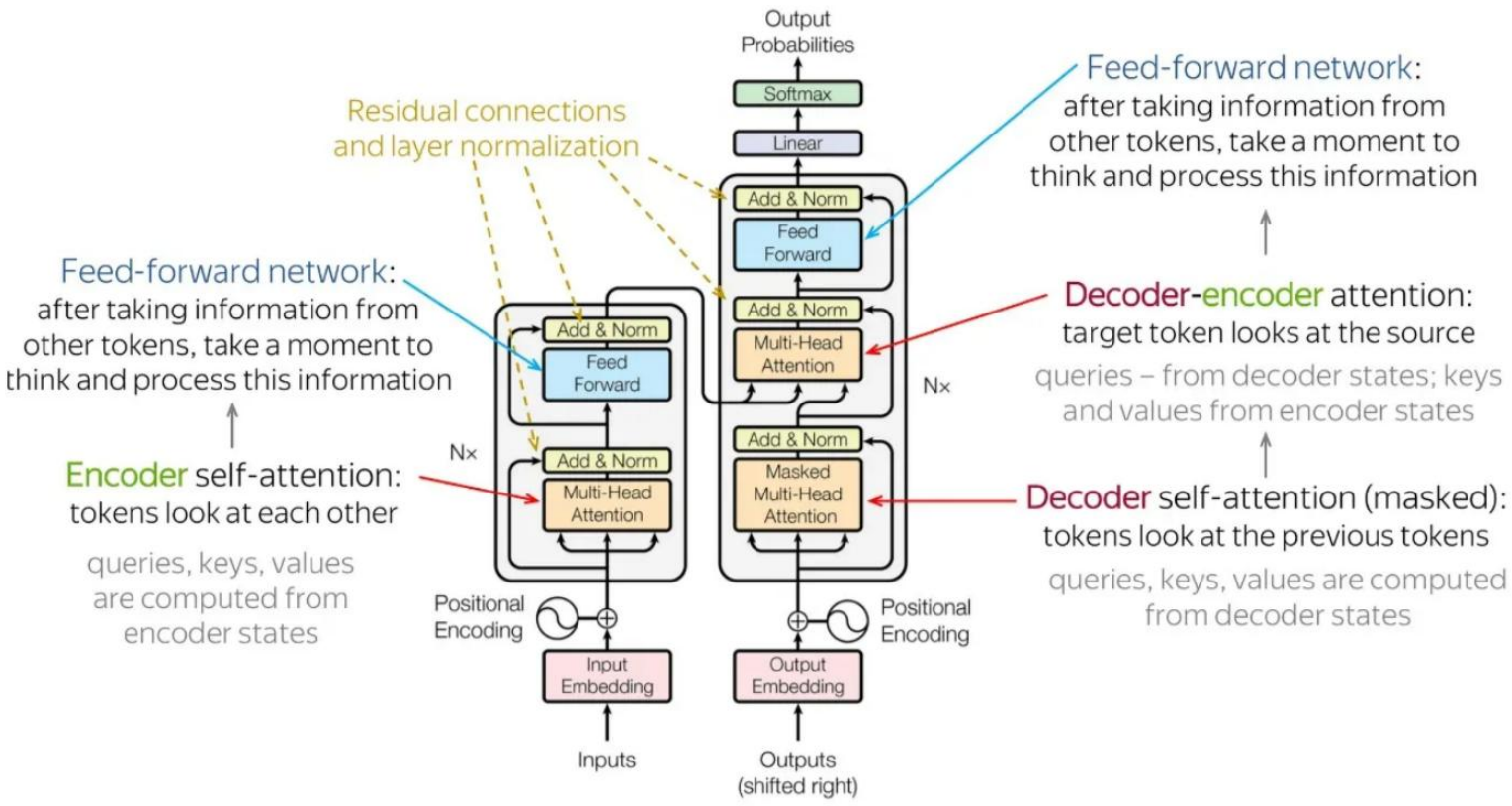
1. **Embeddings**: Convert tokens to vector representations
2. **Positional Encoding**: Adds position information
3. **Multi-Head Attention**: Processes relationships from multiple perspectives
4. **Feed-Forward Networks**: Process each position independently
5. **Layer Normalization**: Stabilizes training
6. **Residual Connections**: Helps with gradient flow

Architecture Variations:

- **Encoder-only** (BERT): Good for understanding (classification, NER)
- **Decoder-only** (GPT): Good for generation
- **Encoder-decoder** (T5): Good for transformation tasks (translation, summarization)



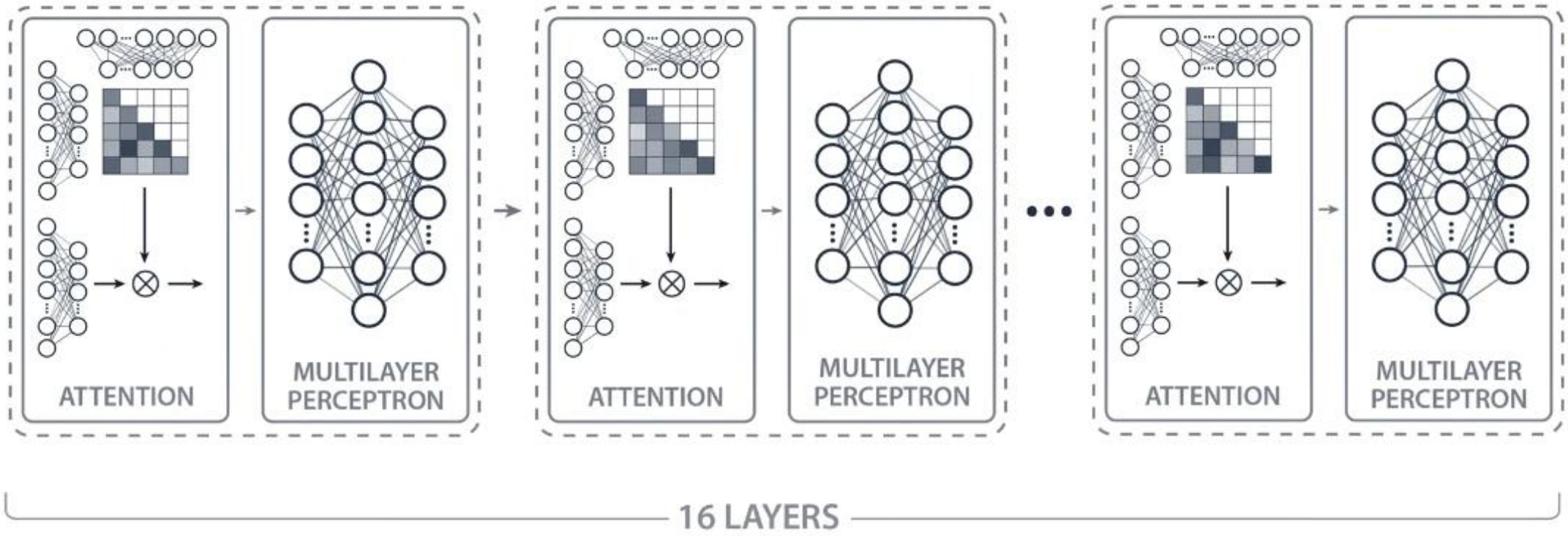
Transformers – Architecture and Principles



Input Sequence

A sequence of words or tokens that the model will process.

Layers of Attention and Feed Forward NN



The Transformer in Detail

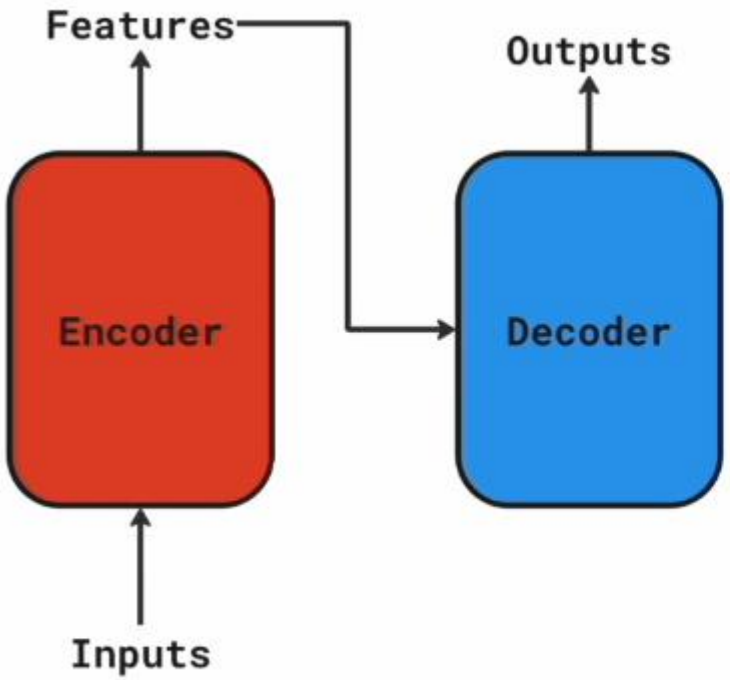
- The transformer architecture is designed to process sequential data, such as natural language, in a highly efficient and effective manner.
- Unlike traditional models that rely on sequential processing (like RNNs), transformers utilize a mechanism called self-attention, allowing them to analyze the entire input sequence at once.
- This capability enables them to capture complex relationships and dependencies between tokens in the sequence.

The transformer model consists of two main parts:

1. **Encoder:** The encoder processes the input sequence and generates a continuous representation of it. This representation captures the contextual information of the input tokens.
2. **Decoder:** The decoder takes the encoder's output and generates the final output sequence. It does this by predicting one token at a time, using the encoded representations and previously generated tokens.

- Both the encoder and decoder are composed of multiple identical layers—typically six layers in the original transformer architecture—allowing for deep learning and complex feature extraction.

Transformer Architecture



"Hello world" -> encoder -> knowledge ->
-> decoder -> *"Hola mundo"*

Key Components of Transformers

1. Multi-Head Attention:

Function: Multi-head attention allows the model to focus on different parts of the input sequence simultaneously.

It computes attention scores for each token in relation to all other tokens, enabling the model to weigh the importance of each token when making predictions.

Mechanism: The attention mechanism uses three vectors: Query (Q), Key (K), and Value (V).

The attention scores are calculated as the dot product of the query and key vectors, scaled by the square root of the dimension of the key vectors.

These scores are then used to weight the value vectors, producing a context-aware representation of the input.

2. Feed-Forward Networks:

Function: Each layer of the encoder and decoder contains a feed-forward neural network that processes the output from the attention mechanism.

This network enhances the model's ability to learn complex representations.

Structure: The feed-forward network consists of two linear transformations with a non-linear activation function (usually ReLU) applied between them.

This allows the model to capture intricate patterns in the data.

Key Components of Transformers

3. Positional Encoding:

Purpose: Since transformers do not inherently understand the order of tokens, positional encodings are added to the input embeddings.

This encoding provides information about the position of each token within the sequence, allowing the model to consider the order of words.

Implementation: Positional encodings are typically generated using sine and cosine functions, which create unique encodings for each position that can be added to the input embeddings.

4. Layer Normalization:

Function: Layer normalization stabilizes the training process by normalizing the inputs to each layer.

This helps mitigate issues related to internal covariate shift and improves convergence during training.

Application: It is applied after the attention and feed-forward layers, ensuring that the outputs are centered and scaled appropriately.

5. Residual Connections:

Purpose: Residual connections help facilitate the flow of gradients during training, addressing the vanishing gradient problem.

They allow the model to learn more effectively by providing a direct path for gradients to flow through the network.

Implementation: The output of each sub-layer (attention and feed-forward) is added back to the original input, creating a shortcut that enhances learning.

The transformer architecture is a powerful and flexible model that has transformed the landscape of natural language processing and other fields. Its ability to process entire sequences simultaneously, leverage self-attention mechanisms, and utilize deep learning through stacked layers makes it a robust choice for a wide range of tasks.

Understanding LLMs

- At a high level, a transformer-based LLM is an autoregressive, **next-token predictor**.
- It takes a sequence of "tokens" (words or parts of words) as input and produces a prediction for what the next token should be. This prediction takes the form of a probability distribution.
- Sampling from this distribution results in the next token. This newly selected token is appended to the sequence, and the entire process is repeated to generate the next token. This loop is repeated until the full response is outputted.
- The model processes text through a pipeline that involves the main components of the transformer:
  - **Tokenizer:** Breaks the input sentence into a list of tokens (words or parts of words).
  - **Embedding layer:** Converts each token into a high-dimensional vector representing its meaning in isolation.
  - **Transformer blocks:** The core of the model, consisting of multiple layers that progressively refine the meaning of the embedding vectors based on their context. Each transformer layer is composed of a self-attention and MLP (multi-layer perceptron) sub-layer.
  - **Unembedding layer & softmax:** Converts the final residual stream vector into a probability distribution over the entire vocabulary. The next token is sampled from this distribution.

Understanding LLMs

- **The residual stream:** This is the central backbone of the transformer. It's a list of vectors (one for each token) that flows through the model. Its shape is the same as the output of the embedding layer. The residual stream starts after the embedding layer and finishes at the unembedding layer.
- **Attention sub-layers:** The primary mechanism for understanding context. Attention heads move information between token positions.
  - They work by having a "query" at a destination token look for relevant information from the "keys" of all previous source tokens.
  - Information from the source tokens' "values" is then copied to the destination, weighted by how much attention was paid.
  - Specialized attention heads can form algorithms, like induction heads, which are crucial for in-context learning by recognizing and completing repeated patterns.
- **MLP sub-layers:** These layers form about two-thirds of the model's parameters and are considered to be its knowledge store.
  - They function as vast key-value memories, where "keys" act as pattern detectors (e.g., "text is about TV shows") and "values" contain the likely next tokens associated with that pattern.

Understanding LLMs

- **The problem of superposition:** LLMs learn far more features (concepts, ideas, patterns) than they have neurons. This forces each neuron to be "polysemantic" and respond to multiple unrelated concepts. This makes it difficult to understand what any single neuron is doing.
- **Solving superposition with sparse autoencoders (SAEs):** To make the model interpretable, SAEs are used to deconstruct the dense, polysemantic activations into a much larger set of sparse, monosemantic features. Each of these new features corresponds to a single, human-understandable concept (e.g., "The Golden Gate Bridge").
- **Circuit tracing and attribution graphs:** This technique goes a step further, aiming to explain how the model reasons. The model's uninterpretable MLP sub-layers are replaced with interpretable transcoders. Attribution graphs trace the flow of information between the monosemantic features within these transcoders, showing the specific sub-network of interconnected features that causally work together to produce a particular output, revealing the "algorithm" the model uses for that task.
- **How do LLMs work?:** Simple analogies like "it's just statistics" or "it's like a computer program" are inadequate explanations of how LLMs work. A better explanation is that LLMs perform tasks by forming emergent circuits. These circuits combine learned statistics, information-moving attention heads, and feature-storing MLP sub-layers into specialized sub-networks that can execute complex behaviors.

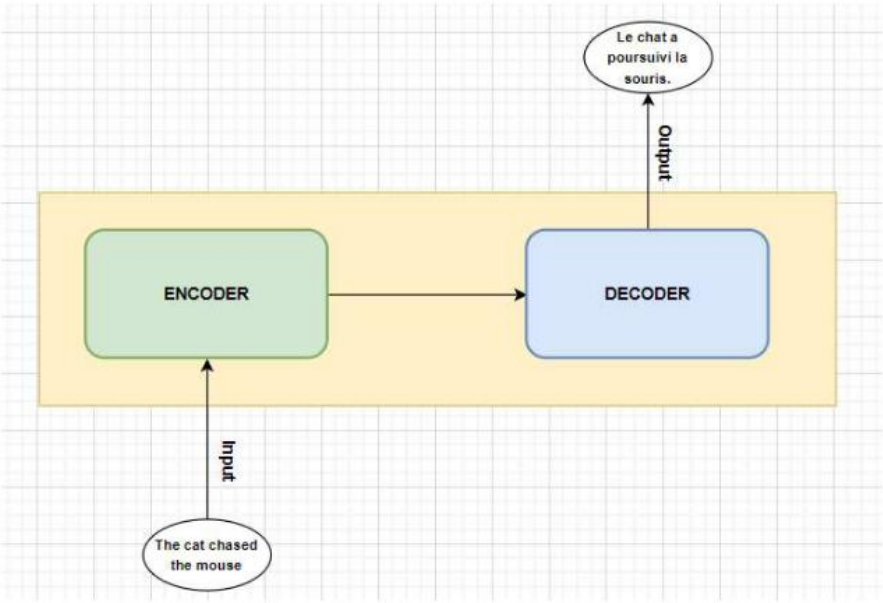
Reference: Understanding LLMs - Insights from Mechanistic Interpretability

Trasnformer Processing Steps

How Transformer Works

Let's understand how the transformer takes input, processes and gives output.
We will consider a simple example of translating an English sentence to French using a transformer model.
Suppose we have,

- **Input sentence:** "Your cat is a lovely cat”
  We want to translate this to French:
- **Output sentence:** "Ton chat est un chat adorable.“
  The transformer takes the input, translates, and gives the output.

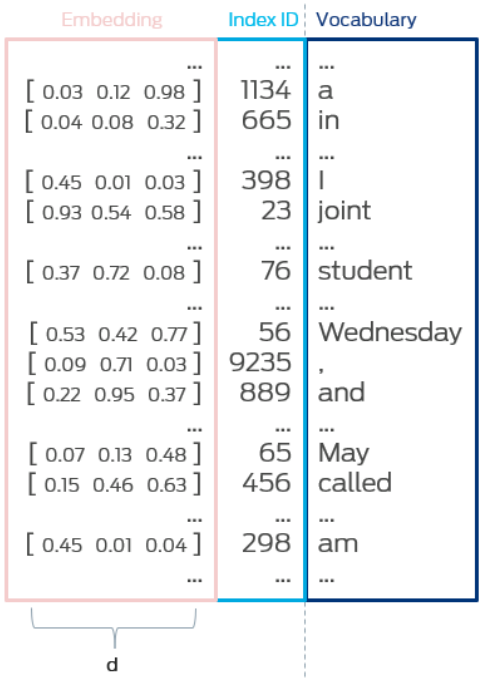


Input Preparation

English sentence (source):

“Your cat is a lovely cat“

- Tokenization: Break the sentence into tokens: ["<s>", "Your", "cat", "is", "a", "lovely", "cat", "</s>"]
- Embedding: Convert each token into a vector using a learned embedding matrix. These embeddings capture the semantic meaning of each word.
- Positional Encoding: Add position-based information to each token embedding to provide information about the position of each token in the sequence. This is necessary because transformers process the entire sequence simultaneously, unlike RNNs that process one word at a time.
- Without positional encoding, the self-attention mechanism would treat the sentence as a bag of words.



Encoder

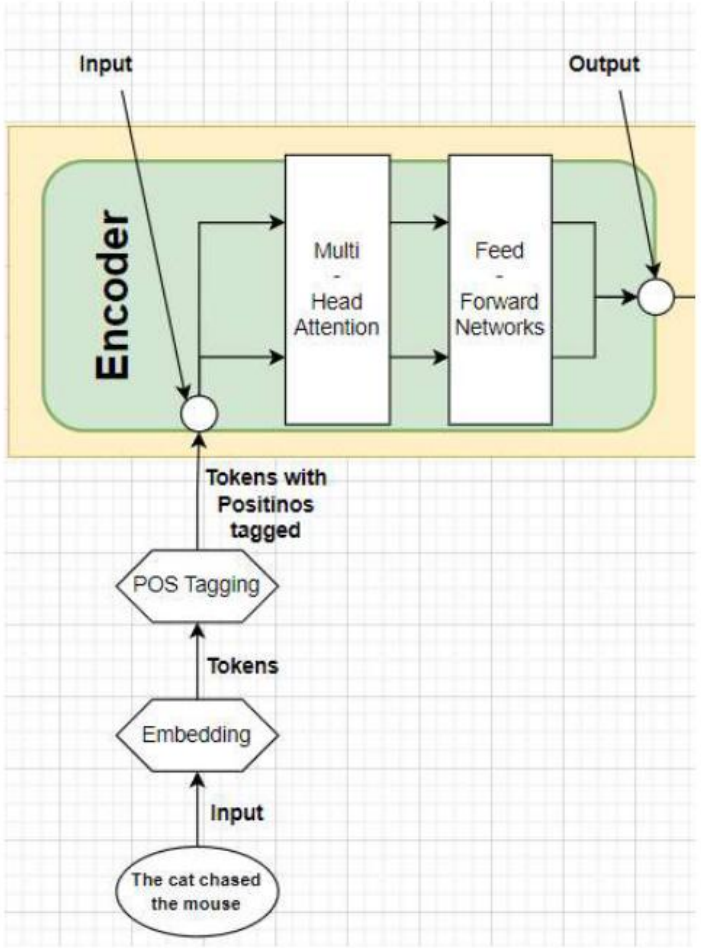
The encoder processes the full input sequence in parallel through a stack of layers.

The encoder takes the input embeddings with positional encodings and passes them through multiple layers of multi-head attention and feed-forward networks. Each encoder layer processes the input, allowing the model to learn complex representations of the sentence.

For example, in a sentence like “The cat chased the mouse”, the attention mechanism in the encoder might learn that "cat" is related to "chased" and "mouse", capturing the semantic relationships between the tokens.

Each encoder layer includes:

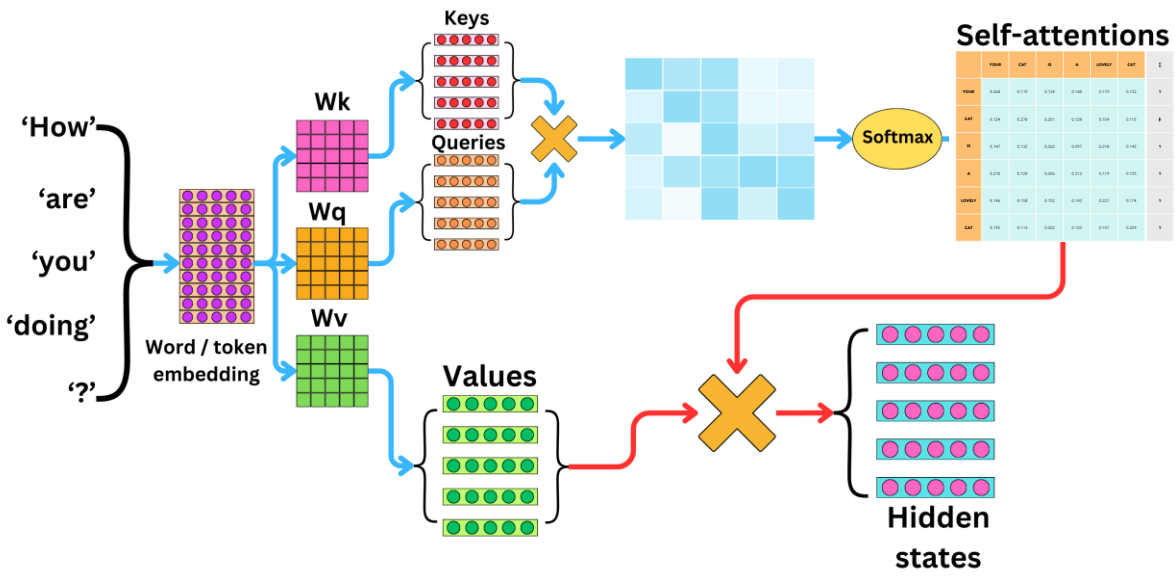
- **Multi-head Self-Attention:**
  Each word learns which other words to focus on.
  Example: The second occurrence of "cat" may attend to the first "cat" to recognize repetition or coreference.
- **Add & Norm:**
  A residual connection followed by layer normalization.
- **Feedforward Network:**
  A two-layer neural network processes each position independently.
- **Add & Norm:**
  Another residual connection and normalization.
- This stack is repeated several times (e.g., 6 layers), producing a set of **contextualized vectors**, one for each token.



The Attention Mechanism

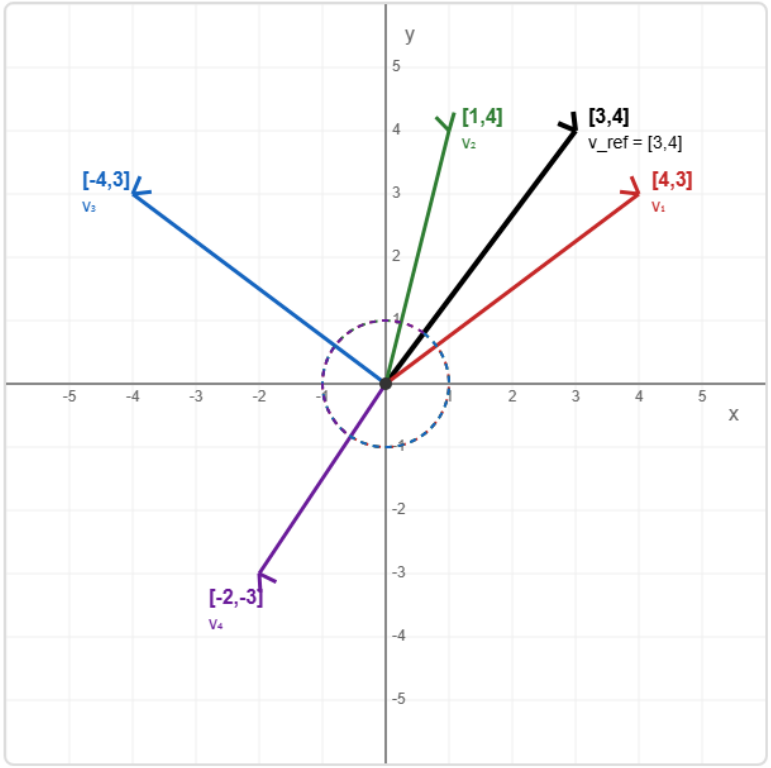
1. **Create Q, K, V matrices**: Each word embedding is multiplied by three learned weight matrices (WQ, WK, WV) to create Query, Key, and Value representations:
   - Q = X · WQ
   - K = X · WK
   - V = X · WV
2. **Compute attention scores**: Each query vector is dot-producted with all key vectors to get raw attention scores:
   - Scores = Q · K^T
3. **Scale the scores**: Divide by √dk (where dk is the dimension of the key vectors) to prevent the values from getting too large:
   - Scaled scores = (Q · K^T) / √dk
4. **Apply softmax**: Convert the scaled scores to probabilities:
   - Attention weights = softmax(Scaled scores)
5. **Compute weighted values**: Multiply the attention weights by the value vectors:
   - Output = Attention weights · V

So the formula is: Attention(Q,K,V) = softmax((Q·K^T)/√dk) · V



Vector Dot Product as Similarity Measure

$$v_1 \cdot v_2 = x_1x_2 + y_1y_2$$



1. Very Similar (High Positive)

`Red Vector v₁ = [4, 3]`

$v_1 \cdot v_ref = (4\times3) + (3\times4) = 24$

Almost same direction as reference

2. Similar (Positive)

`Green Vector v₂ = [1, 4]`

$v_2 \cdot v_ref = (1\times3) + (4\times4) = 19$

Generally same direction

3. Neutral (Zero)

`Blue Vector v₃ = [-4, 3]`

$v_3 \cdot v_ref = (-4\times3) + (3\times4) = 0$

Perpendicular (90° angle)

4. Dissimilar (Negative)

`Purple Vector v₄ = [-2, -3]`

$v_4 \cdot v_ref = (-2\times3) + (-3\times4) = -18$

Opposite direction

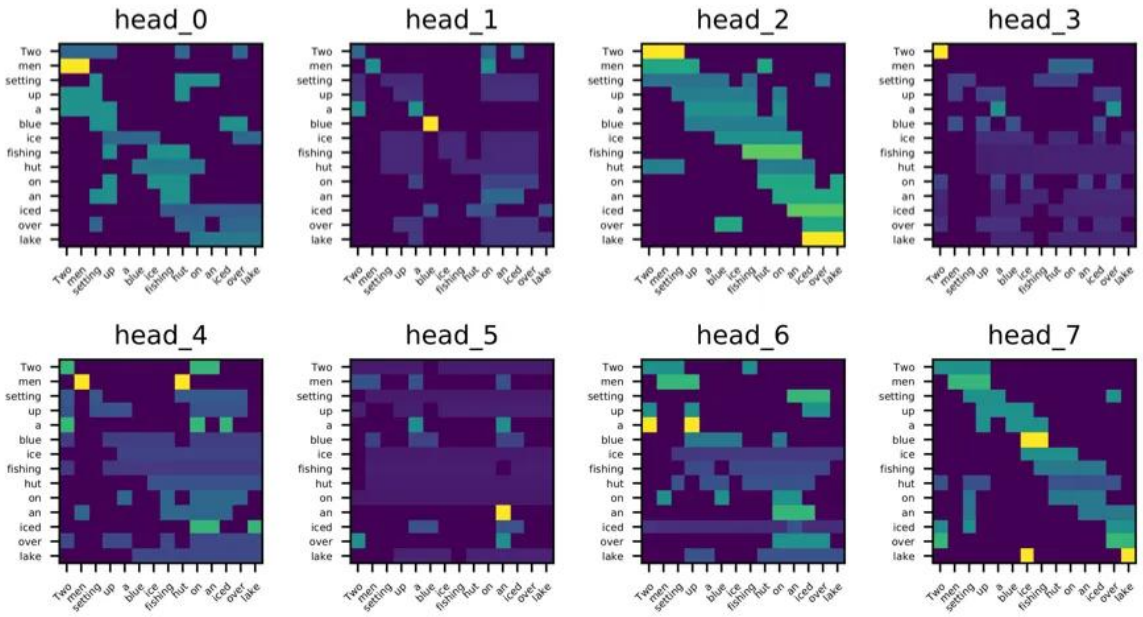
Key Innovation: Self-Attention Mechanism

- Self-attention allows each word to compute its embedding by gathering information from all other words in the sequence.
- The "attention weights" determine how much each word should focus on other words.
- Words that are semantically related tend to have higher attention scores between them.
- This mechanism helps capture long-range dependencies and relationships regardless of word distance.
- Multiple attention heads in parallel (Multi-head Attention) allow the model to focus on different aspects of relationships.

	YOUR	CAT	IS	A	LOVELY	CAT	Σ
YOUR	0.268	0.119	0.134	0.148	0.179	0.152	1
CAT	0.124	0.278	0.201	0.128	0.154	0.115	1
IS	0.147	0.132	0.262	0.097	0.218	0.145	1
A	0.210	0.128	0.206	0.212	0.119	0.125	1
LOVELY	0.146	0.158	0.152	0.143	0.227	0.174	1
CAT	0.195	0.114	0.203	0.103	0.157	0.229	1

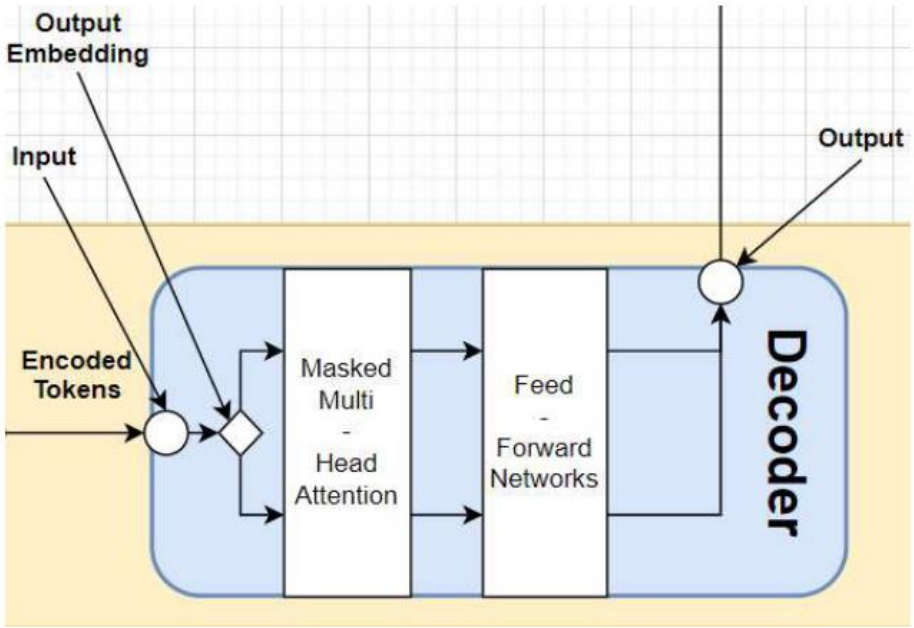
Multiple Attention Heads

- Multiple attention heads in parallel (Multi-head Attention) allow the model to focus on different aspects of relationships.
- For example, one head might learn which words are related by grammar, while another might focus on semantic meaning.
- This allows the model to capture a richer and more comprehensive understanding of the input.



Decoding Process

- The decoder takes the encoder's output and generates the output sequence in French.
- It uses masked multi-head attention to ensure that predictions for a given token do not depend on future tokens.
- This allows the decoder to generate the output one token at a time.
- The decoder also attends to the encoder's output, enabling it to incorporate context from the input sentence while generating the translation.
- For instance, the attention mechanism in the decoder might focus on the representation of "cat" when generating "Le chat", ensuring that the translation is consistent with the input.



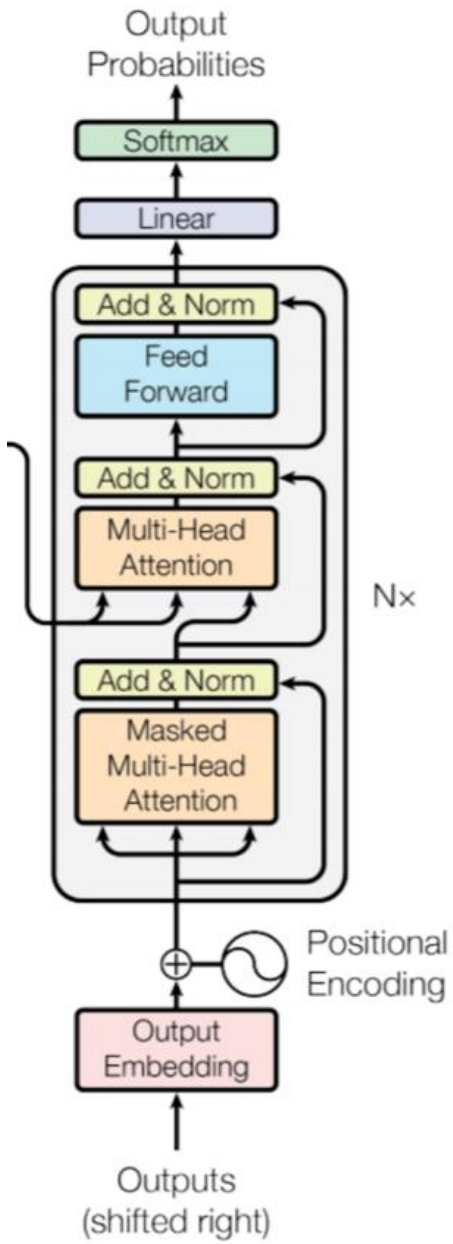
Decoder Components

1. **Masked Multi-head Self-Attention:**
   Looks at the previously generated French words. Future words are masked to prevent cheating.

2. Encoder-Decoder Attention:
Each decoder token can attend to all encoder outputs.
Example: The decoder token "chat" may attend to the English "cat" to align the translation.

3. Feedforward Network:
Processes each token vector separately.

Also, each sub-layer includes residual connections and layer normalization.



Decoder Training and Inference

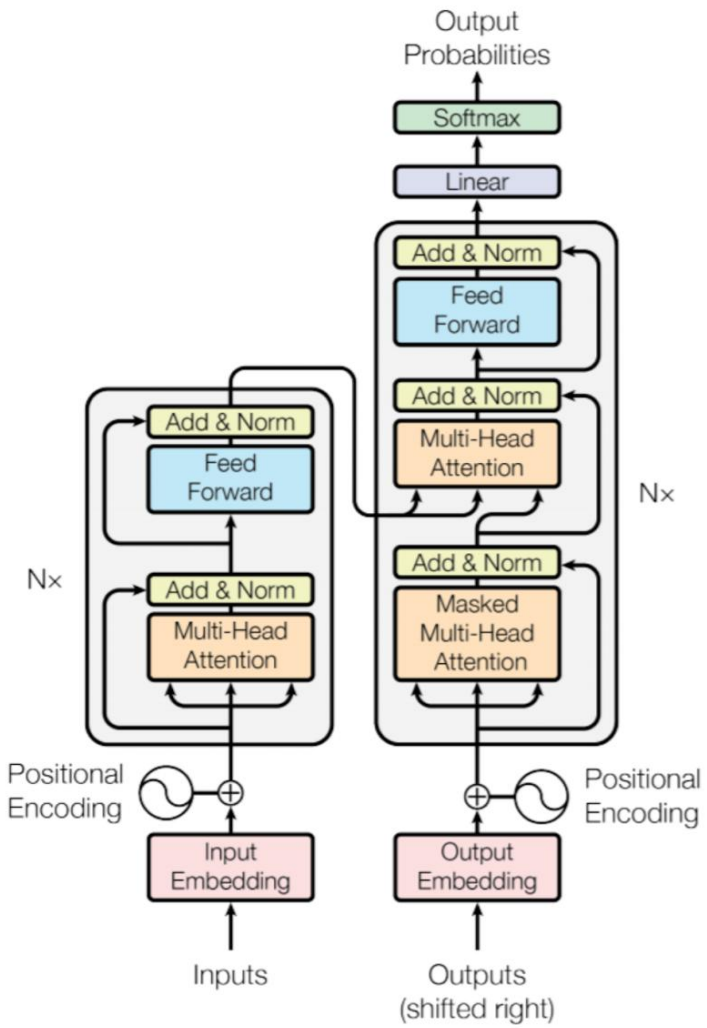
Training Phase:

- The decoder begins with a special start-of-sequence token <s>
- At each time step, it receives the actual previous target words.
- Example: Step 1: <s>, Step 2: <s> Ton, Step 3: <s> Ton chat, etc.

Inference Phase:

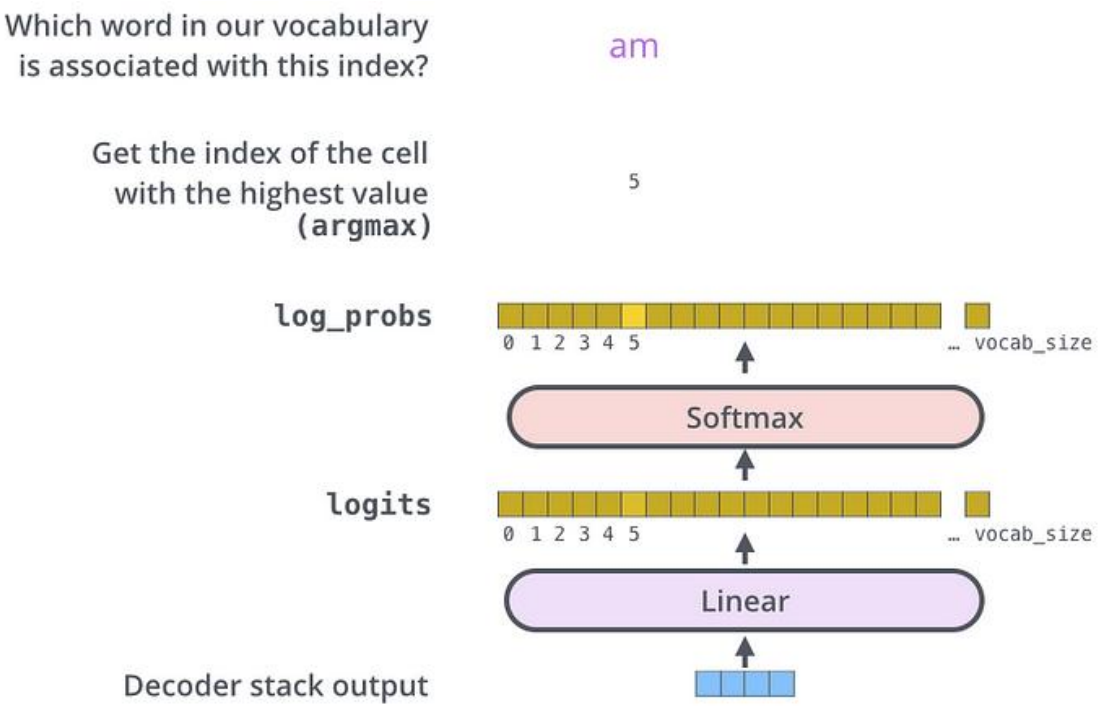
- Starts with <s> and generates one word at a time.
- Each new word is used as input for the next step.
- Example: <s> → Ton → chat → est → ...

Summary:
By combining attention to past outputs and the encoded input, the decoder generates coherent, context-aware text—essential for tasks like translation and summarization.



Output Generation

- The decoder outputs a vector at each time step.
- A linear layer followed by softmax turns this vector into a probability distribution over the French vocabulary.
- The model selects the most probable next word ("Ton", then "chat", then "est", etc.).
- This continues until an end-of-sentence token </s> is generated or a length limit is reached.



A Summary of how the Transformer Works

Input Sentence: "The cat chased the mouse."

Input Encoding:

- Break down the sentence into tokens (words).
- Convert each token into a numerical representation called an embedding.
- Add positional encodings to the embeddings to provide information about the position of each token.

Encoder Processing:

- The encoder takes the input embeddings with positional encodings.
- Pass the input through multiple layers of multi-head attention and feed-forward networks.
- Each encoder layer processes the input, allowing the model to learn complex representations of the sentence.
- The attention mechanism in the encoder learns relationships between tokens (e.g., "cat" is related to "chased" and "mouse").

Decoder Processing:

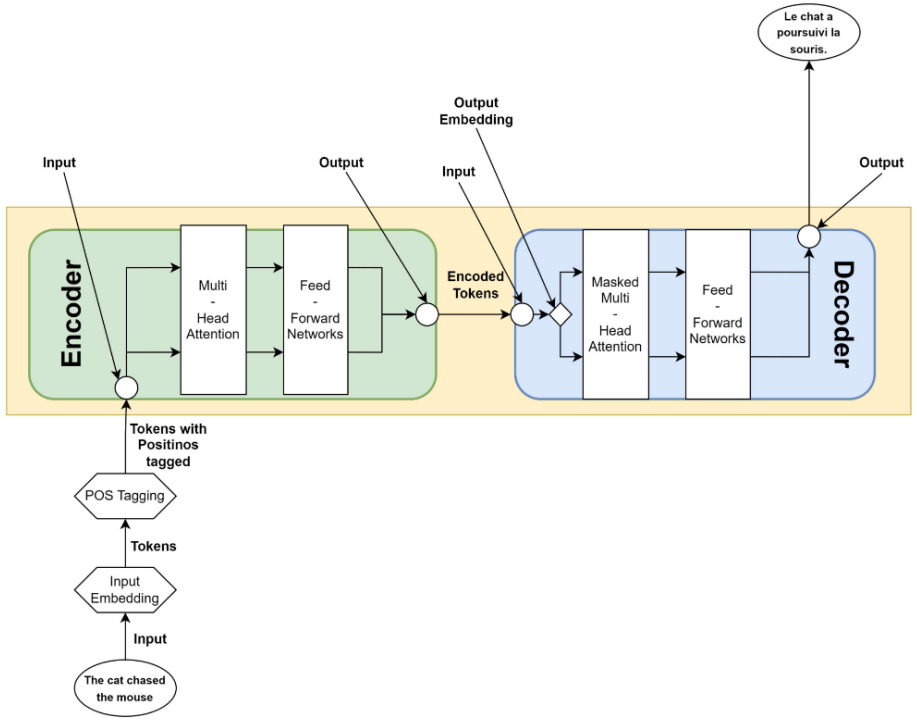
- The decoder takes the encoder's output.
- Use masked multi-head attention to generate the output one token at a time.
- Attend to the encoder's output to incorporate context from the input sentence while generating the translation.
- The attention mechanism in the decoder focuses on relevant parts of the input (e.g., the representation of "cat" when generating "Le chat").

Output Generation:

- The decoder generates the output sequence token by token.
- For the example, it generates: "Le", "chat", "a", "poursuivi", "la", and "souris".
- The complete French translation is: "Le chat a poursuivi la souris."

Key Advantages:

- Process the entire input sequence simultaneously.
- Use attention mechanisms to capture relationships between tokens.
- Efficiently translates sentences, even with long-range dependencies.

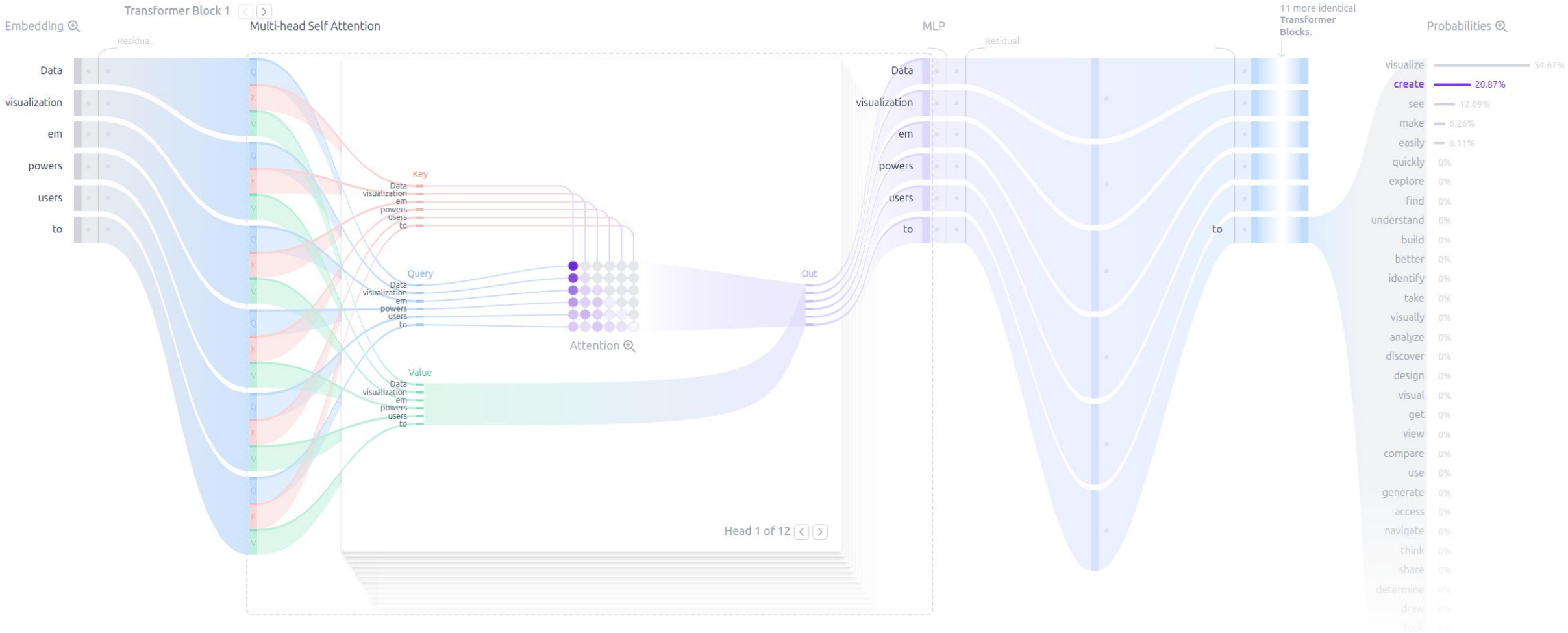


Number of Parameters - Original Transformer Model

Component	Parameter	Formula / Size	Total Parameters
Input	Token embedding	Vocab_Size × d_model = 37000 × 512	≈ 18.94M
	Positional encoding (fixed)	n × d_model	Not learned (original paper used fixed)
Attention (per layer)	Q/K/V weights per head	3 × d_model × d_k = 3 × 512 × 64	98,304
	Output projection	d_model × d_model = 512 × 512	262,144
	Total per Multi-Head block	–	≈ 360K
Feed-Forward (per layer)	Linear 1: 512 × 2048	–	1,048,576
	Linear 2: 2048 × 512	–	1,048,576
	Total FFN per layer	–	≈ 2.10M
LayerNorm	2 × γ, β per layer	2 × d_model = 2 × 512	1,024
Encoder Block Total	–	Attention + FFN + LayerNorm	≈ 2.46M
Encoder Total (6 layers)	–	6 × 2.46M	≈ 14.76M
Decoder Total (6 layers)	Similar structure + cross attention	≈ 2.6M per layer	≈ 15.6M
Output Layer	d_model × Vocab_Size = 512 × 37000	tied/shared with embedding	≈ 18.94M
Total Model Parameters	–	Encoder + Decoder + Embedding + Output	**≈ 65M**

Transformer Explainer

https://poloclub.github.io/transformer-explainer/



References

- https://jalammar.github.io/illustrated-transformer/
- https://tamoghnasaha-22.medium.com/transformers-illustrated-5c9205a6c70f

Context Aware Embeddings

Contextual Word Embeddings (BERT and GPT)

Key Innovation

Unlike static embeddings (Word2Vec, GloVe), contextual models generate **different vectors for the same word** depending on its context.

Approach

- Uses deep, pre-trained neural networks (often transformer-based)
- Embeddings are derived from entire sentences, capturing syntax and semantics dynamically

Examples

- **BERT - Bidirectional Encoder Representations from Transformers (2018)**: Transformer-based neural networks trained with masked language modeling and next sentence prediction
- **GPT - Generative Pre-training Transformer (2018):** Transformer-based unidirectional language model focused on generation.

Characteristics

- Embeddings are **context-sensitive** (e.g., “bank” in “river bank” vs. “savings bank”)
- Each word is embedded based on its role in the sentence.
- Embeddings vary for the same word depending on its position and meaning.
- Significantly improve performance on downstream NLP tasks.

BERT: Bidirectional Encoder Representations from Transformers

- Developed by Google in 2018
- A **pre-trained language model** based on the **Transformer encoder**
- Reads text **bidirectionally**, enabling deep contextual understanding

Key Ideas

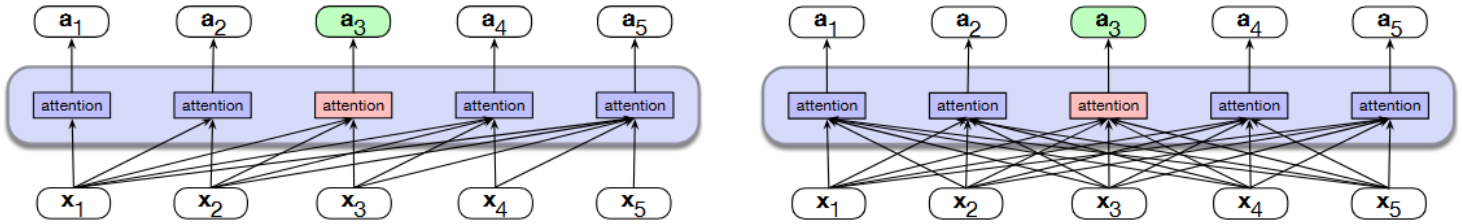
- Uses only the **encoder** stack of the Transformer
- Pre-trained on large text corpora, then fine-tuned on specific tasks

Pretraining Objectives

- **Masked Language Modeling (MLM)**: Predict randomly masked words in a sentence
- **Next Sentence Prediction (NSP)**: Predict if one sentence follows another

Applications

- Sentiment Analysis
- Question Answering
- Named Entity Recognition
- Text Classification
- Semantic Search



a) A causal self-attention layer

b) A bidirectional self-attention layer

More About BERT

Key Advantages

- Contextual embeddings: Word meanings change based on context
- Captures long-range dependencies
- Pre-trained on massive datasets → Transfer learning
- State-of-the-art performance on 11 NLP tasks when released

How BERT Works:

- **Multiple stacked encoder layers** with self-attention mechanisms
- **No decoders** - purely focused on understanding input
- **Captures relationships** between words and their context
- **Powerful context learning** - understands word relationships in sentences
- **Meaningful representations** - transforms text into useful numbers

Architecture Overview

```
Input: [CLS] The cat sat on the mat [SEP]
            ↓
    Token Embeddings
    Position Embeddings
    Segment Embeddings
            ↓
    Transformer Encoder (12 or 24 layers)
            ↓
    Contextual Representations
```

Key Components

- **[CLS]**: Classification token (sentence-level tasks)
- **[SEP]**: Separator token (between sentences)
- **Multi-head attention**: Allows model to focus on different positions
- **Feed-forward networks**: Process attention outputs

BERT's Training Data

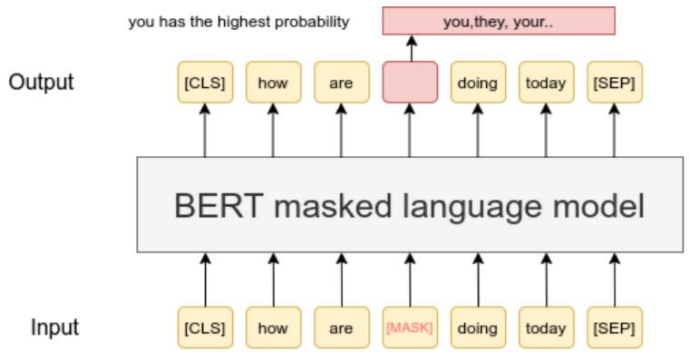
Pre-training Foundation

- **Training corpus**: 3+ billion words
  - English Wikipedia
  - ~10,000 unpublished books
- **Clean, large-scale dataset** for comprehensive language learning
- **Pre-training approach** to learn language patterns and context

Training Objective 1 - Masked Language Modeling

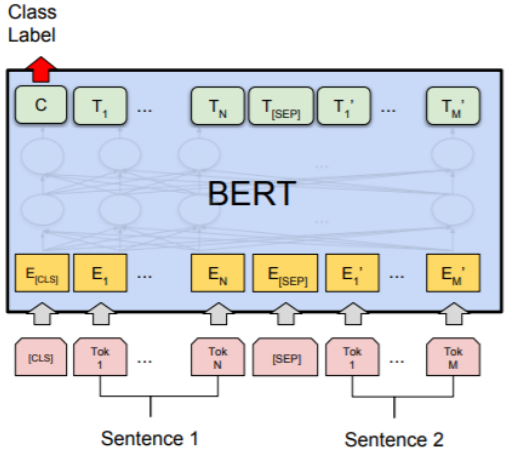
Learning Through "Fill in the Blanks"

- **Random word masking**: Some words are hidden during training
- **Context-based prediction**: BERT predicts masked words using surrounding context
- **Example**: "I love eating _____ in my fruit salad" → "lemons"
- **Key benefit**: Learns word relationships and contextual understanding
- **Powerful concept**: Used in many other AI applications



Training Objective 2 - Next Sentence Prediction

- **Title: Understanding Sentence Relationships**
- **Sentence pair analysis**: Given two sentences, determine if B follows A
- **Logical connection assessment**: Analyzes context and content
- **Example**:
  - A: "The cat climbed the tree"
  - B: "It was trying to catch a bird" ✓ (follows logically)
  - C: "The weather is nice today" ✗ (unrelated)
- **Note**: Later removed in newer models (MLM proved sufficient)



(a) Sentence Pair Classification Tasks: MNLI, QQP, QNLI, STS-B, MRPC, RTE, SWAG

BERT's Ideal Use Cases

Where BERT Excels

- **Text Classification**: Sentiment analysis, topic classification, spam detection
- **Named Entity Recognition**: Identifying people, organizations, locations, dates
- **Extractive Question Answering**: Finding answers within provided context
- **Semantic Similarity**: Measuring similarity between sentences/paragraphs
- **Key advantage**: Perfect for understanding tasks, not generation

Extractive Question Answering Example

How BERT Answers Questions:

- **Process**: Analyzes both context and question
- **Method**: Identifies start and end tokens of the answer
- **Example**:
  - Context: "Mount Everest is the highest mountain..."
  - Question: "What is the highest mountain?"
  - Answer: "Mount Everest" (extracted, not generated)
- **Important**: BERT extracts existing text, doesn't generate new content

Component	BERT-Base (L=12, H=768, A=12)	BERT-Large (L=24, H=1024, A=16)
Embedding Layer - Token + Positional + Segment	30522 × 768 + 512 × 768 + 2 × 768 ≈ 23.8M	30522 × 1024 + 512 × 1024 + 2 × 1024≈ 31.4M
Self-Attention - Q/K/V + Output per layer	$4 \times H^2 = 4 \times 768^2 = 2.36M$	$4 \times 1024^2 = 4.19M$
Total across all layers	12 × 2.36M = 28.3M	24 × 4.19M = 100.6M
Feedforward - Two linear layers per layer	768 × 3072 + 3072 × 768 = 4.71M	1024 × 4096 × 2 = 8.39M
Total across all layers	12 × 4.71M = 56.5M	24 × 8.39M = 201.4M
LayerNorms - Two per layer	2 × 768 = 1.5K × 12 = 18K	2 × 1024 × 24 = 49K
Pooler - Final CLS output to 768 or 1024	768 × 768 = 0.59M	1024 × 1024 = 1.05M
Total Parameters	**≈ 110M**	**≈ 340M**

- **L: number of layers (Transformer blocks)**
- **H: hidden size**
- **A: number of attention heads (H / A = size per head)**
- **Vocabulary size: 30,522**
- **FFN hidden size: 4×H (3072 for base, 4096 for large)**

BERT Variants

Common BERT Models

Model	Parameters	Layers	Hidden Size	Attention Heads
BERT-Base	110M	12	768	12
BERT-Large	340M	24	1024	16
DistilBERT	66M	6	768	12
RoBERTa	355M	24	1024	16
ALBERT	12M-235M	12-24	768-4096	12-64

Specialized Variants

- **BioBERT**: Biomedical text
- **SciBERT**: Scientific text
- **FinBERT**: Financial text
- **ClinicalBERT**: Clinical notes

Example: Print out BERT Embeddings

```
from transformers import BertTokenizer, BertModel
import torch

# Load pretrained BERT
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertModel.from_pretrained('bert-base-uncased’)

# Sentence
sentence = "He went to the bank to deposit money.“

# Tokenize
inputs = tokenizer(sentence, return_tensors='pt’)

# Get outputs
with torch.no_grad():  # No training, just inference
    outputs = model(**inputs)

# Get the hidden states (embeddings)
embeddings = outputs.last_hidden_state  # Shape: (batch_size, sequence_length,

# (hidden_size)
print(embeddings.shape)  # Example output: torch.Size([1, 11, 768])
```

Open in Colab

GPT – Overview and Architecture

What is Generative **P**re-trained **T**ransformer (GPT)?

- A family of **Transformer-based language models** developed by OpenAI
- Uses only the **decoder stack** of the original Transformer architecture
- Trained with **causal (autoregressive) language modeling** to predict the next token
- Focuses on generating human-like text (unlike BERT's understanding focus)

Training Objective

- Predict the next token in a sequence

GPT Variants

- **GPT-1 (2018, 120M Parameters)**: Introduced the pretrain-then-finetune paradigm
- **GPT-2 (2019, 1.5B Parameters)**: Scaled up model size, trained on web-scale data, Last publicly available weights
- **GPT-3 (2020, 175B Parameters)**: Enabled in-context learning, 100x boost, ~800GB file size
- **GPT-4 (2023, ~1T parameters (estimated))**: Multimodal, stronger reasoning and generalization

Applications (Wide range of NLP tasks through text generation)

- Text generation (e.g., chat, storytelling, code)
- Summarization
- Translation
- Question answering
- Semantic search and reasoning tasks

GPT's Architecture Deep Dive

How GPT Works

- **Multiple stacked decoder layers** with self-attention mechanisms
- **No encoders** - purely focused on text generation
- **Unidirectional processing** - considers only left-side context
- **Next word prediction** based on preceding words
- **High-quality text generation** from learned context patterns
- **Contextual understanding** combined with text creation capability

GPT Training Data

Pre-training Foundation

- **Massive, diverse datasets** including:
  - Web pages
  - Books and articles
  - Billions of words total
- **Different datasets** for each GPT version
- **Large-scale exposure** to language patterns and context
- **Quality varies** but emphasis on diversity and scale

Causal Language Modeling

GPT's Single Training Objective

- **Core task**: Predict the next word in a sequence
- **Method**: Uses context from preceding words only
- **Example**: "I love drinking fresh ___" → "lemonade"
- **Key benefits**:
  - Learns word relationships and context
  - Develops language patterns
  - Enables coherent text generation
- **Simplicity**: No masked language modeling or next sentence prediction

GPT's Ideal Use Cases

Where GPT Excels

- **Text Generation**: Story creation, creative writing, conversations
- **Instruction Following**: Responding to prompts and commands
- **Translation**: Converting text between languages
- **Summarization**: Creating concise summaries of longer texts
- **Code Generation**: Programming assistance and code completion
- **Versatile Applications**: Any task involving text output

Number of Parameters – GPT 3

Component	Parameter	Formula / Size	Total Parameters (Approx.)
Embedding Layer	Token Embeddings	Vocab × d_model = 50K × 12288	614.4M
	Positional Embeddings	n_ctx × d_model = 2048 × 12288	25.2M
Self-Attention (per layer)	Q, K, V, Output	4 × d_model × d_model = 4 × 12288^2	604.6M per layer
Feedforward (per layer)	2 layers (gelu)	d_model × d_ff + d_ff × d_model	1.2B per layer
LayerNorms (per layer)	Two per layer	2 × d_model	24.6K per layer
Total per layer	–	Self-attn + FFN + norms	≈ 1.8B per layer
Transformer Block Total	96 × 1.8B	–	≈ 172.8B
Final LayerNorm	–	d_model	12.3K
Output Layer (tied)	Shared with token embedding	–	— (tied with input embedding)
Total Parameters	–	Sum of above	**≈ 175B**

- **d_model = 12288**
- **n_layers = 96**
- **n_heads = 96 → each head has d_k = d_v = 128**
- **d_ff = 4 × d_model = 49152**
- **Vocabulary size ≈ 50,000**
- **Sequence length (n_ctx) = 2048**

GPT vs BERT Architecture

Aspect	GPT	BERT
Architecture	Decoder-only	Encoder-only
Text Processing	Unidirectional (left-to-right)	Bidirectional
Primary Goal	Text generation	Text understanding
Context	Previous words only	All surrounding words
Use Cases	Generation tasks	Classification/extraction

What is Hugging Face? 🤔

- **A company and a community platform** focused on democratizing Artificial Intelligence, especially Natural Language Processing (NLP) and Machine Learning (ML).
- Often called the "**GitHub for Machine Learning**."
- **Mission**: To make state-of-the-art ML models, datasets, and tools accessible to everyone.
- Started in 2016, initially with a chatbot app, then pivoted to open-source ML.

What does hugging face provide?

1. **Accessibility**: Provides easy access to thousands of pre-trained LLMs.
2. **Standardization**: Offers standardized tools and interfaces for working with different models.
3. **Collaboration**: Fosters a vibrant community for sharing models, datasets, and knowledge.
4. **Innovation**: Accelerates research and development in the LLM field.
5. **Ease of Use**: Simplifies complex ML workflows, from data preparation to model deployment.

Core Components of the Hugging Face Ecosystem

- **Hugging Face Hub**:
  - The central place to find, share, and collaborate on models, datasets, and ML applications (Spaces).
  - Over 1.7 million models, 75,000+ datasets!
- **Transformers Library**:
  - Python library providing thousands of pre-trained models for NLP, Computer Vision, Audio, and more.
  - Supports PyTorch, TensorFlow, and JAX.
  - Makes downloading, training, and using state-of-the-art models incredibly simple.
- **Datasets Library**:
  - Efficiently load and process large datasets.
  - Optimized for speed and memory, built on Apache Arrow.
  - Access to a vast collection of public datasets.
- **Tokenizers Library**:
  - Provides high-performance tokenizers crucial for preparing text data for LLMs.
  - Offers various tokenization algorithms and pre-trained tokenizers.

The Model Hub

Over 1,700,000+ Models Available

Popular Model Categories:

- **Text Generation**: GPT, LLaMA, Mistral, CodeLlama
- **Text Classification**: BERT, RoBERTa, DeBERTa
- **Question Answering**: BERT-based models
- **Translation**: T5, mT5, NLLB
- **Code Generation**: CodeT5, StarCoder
- **Multimodal**: CLIP, BLIP, LLaVA

Getting Started with Hugging Face

- Explore the Hub: huggingface.co
- Browse models, datasets, and Spaces.
- Install Libraries:

```
pip install transformers datasets tokenizers
accelerate gradio
```

- Try a Pipeline:

```
# Example: Sentiment Analysis

from transformers import pipeline

classifier = pipeline("sentiment-analysis")

result = classifier("Hugging Face is awesome!")

print(result)

# Example: Text Generation

generator = pipeline("text-generation")

output = generator("In a world of large language models,",
   max_length=50)

print(output)
```

Under the Hood

1. Automatic model selection
2. Tokenization handled
3. Inference optimization
4. Result formatting
5. Device management

Traditional Approach

1. Load tokenizer
2. Preprocess text
3. Load model
4. Run inference
5. Post-process results

... 50+ lines of code

Other Hugging face Pipelines

The Hugging Face `transformers` library supports a wide range of **pipelines**, each designed for a specific **natural language processing (NLP)** or **vision** task — so you can use powerful models without deep setup.

Pipeline Name	Task Description
"sentiment-analysis"	Classify sentiment (positive/negative)
"text-classification"	General text classification (multi-label or multi-class)
"zero-shot-classification"	Classify into labels **without training** on them
"text-generation"	Generate text (e.g., GPT models)
"text2text-generation"	Text-to-text tasks (e.g., summarization, translation)
"translation"	Translate between languages
"summarization"	Generate a summary of input text
"question-answering"	Extract answer from context
"fill-mask"	Predict missing word in a sentence (BERT-style)
"ner" (Named Entity Recognition)	Extract entities (like names, places, etc.)
"conversational"	Chatbot-style conversation
"sentence-similarity"	Measure similarity between two sentences
"token-classification"	Classify each token (used for NER, POS tagging, etc.)
"feature-extraction"	Extract embeddings/features from a model
"table-question-answering"	QA over structured data (tables)

➤ **Sentiment Analysis**

python Copy Edit

```python
pipeline("sentiment-analysis")("I love this!")
```

➤ **Summarization**

python Copy Edit

```python
pipeline("summarization")("Long article text goes here...")
```

➤ **Translation**

python Copy Edit

```python
pipeline("translation_en_to_fr")("This is amazing.")
```

➤ **Question Answering**

python Copy Edit

```python
qa = pipeline("question-answering")
qa({
    "question": "Where do pandas live?",
    "context": "Pandas are native to China and prefer bamboo forests."
})
```

To list all available pipelines in code:

python Copy Edit

```python
from transformers.pipelines import SUPPORTED_TASKS
print(SUPPORTED_TASKS.keys())
```

Open in Colab

Training LLMs

What is LLM Training?

- LLM Training is the process of teaching a neural network to understand, generate, and manipulate human language.
- It involves feeding the model vast amounts of text data.
- The model learns patterns, grammar, context, and even some level of "knowledge" from this data.
- The goal is to adjust the model's internal parameters (weights and biases) to perform specific language tasks effectively.
- Modern models have BILLIONS of parameters (numbers) to learn during the training process.

Essential Components of LLM Training

1. **Dataset**: Large corpus of text data (e.g., books, articles, websites). Quality, quantity, and diversity are crucial.
2. **Model Architecture**: The neural network structure, predominantly the Transformer architecture (with self-attention mechanisms).
3. **Loss Function**: A function that measures the difference between the model's predictions and the actual target values (e.g., cross-entropy for next-word prediction).
4. **Optimizer**: An algorithm that updates the model's parameters to minimize the loss function (e.g., Adam, SGD).

The General Training Loop

1. **Data Preparation**: Collecting, cleaning, and tokenizing the text data into a format the model can understand.
2. **Model Initialization**: Setting initial random values for the model's parameters.
3. **Forward Pass**: Feeding input data through the model to get predictions.
4. **Loss Calculation**: Comparing predictions to the actual data to quantify error using the loss function.
5. **Backward Pass (Backpropagation)**: Calculating gradients, which indicate how each parameter contributed to the error.
6. **Parameter Update**: Adjusting model parameters using the optimizer in the direction that reduces the loss.
7. **Iteration**: Repeating steps 3-6 for many batches of data over multiple epochs (passes through the entire dataset).

Main Training Phases

1. LLM Pretraining.
2. LLM Fine Tuning

Pre-training: Building the Foundation

- The initial, most resource-intensive training phase.
- Models are trained on massive, diverse datasets (e.g., Common Crawl, Wikipedia, books).
- The objective of this step is to learn general language understanding, grammar, common sense reasoning, and factual knowledge.
- Usually self-supervised (e.g., predicting masked words, next sentence prediction).
- Results in a **base model** with broad capabilities.
- Examples: GPT-3, BERT, Llama pre-training.

Fine-tuning: Specializing the Model

- Takes a pre-trained base model and further trains it on a smaller, task-specific dataset.
- The objective of this step is to adapt the general knowledge of the pre-trained model to perform well on a particular downstream task (e.g., medical question answering, legal document summarization).
- It requires significantly less data and computation than pre-training.
- Can also be used for instruction tuning (following prompts) or aligning with human preferences (RLHF).

Data: The Critical Ingredient

- **Quantity**: LLMs require vast amounts of text to learn effectively. "More data is better" is often true, up to a point.
- **Quality**: Clean, well-formatted, and coherent data leads to better models. Garbage in, garbage out.
- **Diversity**: Exposure to various styles, domains, and perspectives helps create more robust and less biased models.
- **Preprocessing**:
  - **Tokenization**: Breaking text into smaller units (words, sub-words).
  - **Normalization**: Standardizing text (e.g., lowercasing, removing special characters).
  - Creating input IDs, attention masks.

Model Architecture: The Transformer

- The dominant architecture for state-of-the-art LLMs.
- Key Innovations:
  - **Self-Attention Mechanism**: Allows the model to weigh the importance of different words in a sequence when processing information, capturing long-range dependencies.
  - **Positional Encodings**: Injects information about the position of tokens in the sequence.
  - **Encoder-Decoder Structures** (for some tasks) or **Decoder-Only Structures** (common for generation).
  - **Feed-Forward Networks**: Applied independently to each position

Computational Demands & Challenges

- **Hardware**: Requires powerful GPUs (Graphics Processing Units) or TPUs (Tensor Processing Units) for parallel computation.
- **Distributed Training**: Often necessary to train large models across multiple GPUs or machines, adding complexity.
- **Time**: Pre-training can take weeks or months, even with significant computational resources.
- **Cost**: Significant expenses for hardware, cloud computing, and energy consumption.
- **Memory**: Model parameters and activations require substantial memory. Techniques like mixed-precision training help.

Evaluating Trained LLMs

- **Perplexity**: Measures how well a probability model predicts a sample. Lower is better.
- **Task-Specific Metrics**:
  - **BLEU, ROUGE**: For translation and summarization (overlap with reference texts).
  - **Accuracy, F1-score**: For classification tasks (e.g., sentiment analysis).
- **Benchmarks**: Standardized datasets and tasks for comparing models (e.g., GLUE, SuperGLUE, MMLU).
- **Human Evaluation**: Assessing fluency, coherence, helpfulness, and harmlessness by human raters. Often crucial for real-world performance.

Ethical Considerations in Training

- **Bias Amplification**: Models can learn and perpetuate biases present in the training data (e.g., gender, racial, societal biases).
- **Harmful Content Generation**: Potential to generate misinformation, hate speech, or other harmful text.
- **Data Privacy**: Ensuring that sensitive information from training data is not memorized or leaked.
- **Environmental Impact**: Significant energy consumption of training large models.
- **Accessibility and Equity**: Ensuring benefits of LLMs are widely accessible.

Memory Usage During Training

MODEL PARAMETERS

The **weights** of the mode. Assuming **16-bit half precision**, a model with **N billion parameters** requires **2*N GB** of memory for the weights alone

GRADIENTS

Gradients of the loss function relative to each model parameter are calculated. These gradients, being the same size and type as the model parameters, also occupy **2*N GB** of memory.

OPTIMIZER STATES

Optimizers, such as Adam, maintain state information for each parameter, typically requiring storage of two additional values per parameter. This can double the memory used by optimizer states to between **4-8*N GB**, depending on the precision.

TRAINING DATA

The **training data** itself varies significantly but includes at least one batch in memory, influenced by **batch size**, and the memory size of each data element depends on its **sequence length and embedding size**.

LLM Training Details - More Recent Models & Considerations

Model (Version/Size)	Est. Data Size (Tokens)	Context Size (Max Tokens)	Est. GPUs / Compute	Est. Training Time	Est. Electricity / Carbon Footprint	Est. Training Cost
GPT-3 (Base models like Davinci)	~500 Billion - 1 Trillion (incl. C4, Wikipedia, Books, etc.)	2,048 (later models up to 4,096)	~10,000 V100 GPUs (for original run) / ~3.6M A100-equivalent hours (PaLM 540B reference)	~34 days (using 1,024 A100s, research estimate) - Several months (actual, unconfirmed)	~1,287 MWh (training) / ~552 tons CO_2eq (Patterson et al.)	$4.6M - $12M+ (compute, various estimates)
Llama 2 (All sizes)	2 Trillion	4,096	Reported 6,000 GPU-months (A100-80GB equivalent for the family) / 1.7M+ GPU hours for 70B	Jan 2023 - July 2023 (overall project, specific model run time shorter within this)	3.3M kWh (entire project) / 539 tons CO_2eq (training, 100% offset by Meta)	Significant (part of Meta's AI investment)
BLOOM (176B)	366 Billion (1.6 TB)	2,048	384 A100 GPUs	~3.5 - 4 months	~433 MWh (training) / ~25-55 tons CO_2eq (trained in France, low-carbon energy)	~$2M - $5M (compute, public estimates)

LLM Training Details - More Recent Models & Considerations

Model (Version/Size)	Est. Data Size (Tokens)	Context Size (Max Tokens)	Est. GPUs / Compute	Est. Training Time	Est. Electricity / Carbon Footprint	Est. Training Cost
GPT-4	Not officially disclosed (speculated >> GPT-3, likely multi-trillion)	8,192 (GPT-4-8k) & 32,768 (GPT-4-32k); GPT-4 Turbo: 128,000	Not officially disclosed (speculated tens of thousands of A100s/H100s)	~5-6 months (speculative estimates)	Not disclosed (Expected to be significantly higher than GPT-3; estimates range from 20,000-78,000 MWh & thousands of tons CO_2eq for comparable efforts)	Est. >$60M - $100M+ (compute, speculative)
Llama 3 (Instruct models)	>15 Trillion (for the Llama 3 family)	8,192 (some reports suggest up to 128k for future/experimental versions)	Significant clusters of H100s (e.g., Meta mentioned two 24k H100 clusters)	~3 days (8B), ~17 days (70B), ~97 days (est. for 400B+ on 16k H100s)	Llama 3.1 405B est. ~11 GWh. Carbon footprint not yet fully disclosed, but Meta aims for net-zero operations.	Very High (part of Meta's large AI infrastructure investment)
Gemini 1.0 (Pro/Ultra)	Not officially disclosed (multimodal, likely vast & diverse datasets)	32,768 (Gemini 1.0 Pro); Gemini 1.5 Pro: 1 Million (up to 10M experimental)	Trained on Google's TPU v4 and v5e pods (thousands to tens of thousands of TPUs)	Not publicly disclosed (likely months)	Not disclosed. Google emphasizes efficiency & use of renewable energy. Gemini 1.0 was reported to be more efficient than some predecessors.	Very High (part of Google DeepMind's core AI efforts)

Discussion / Q&A

- What are the biggest challenges in training even larger and more capable LLMs?
- How can we mitigate biases in LLM training data and subsequent models?
- What future advancements in LLM training do you foresee?

Fine-Tuning Large Language Models (LLMs)

What is Fine-Tuning?

- **Pre-training Phase**
  - Trained on large corpus using Masked Language Modeling (MLM) and Next Sentence Prediction (NSP)
- **Fine-Tuning Phase**
  - Fine-tuning is the process of continuing the training of a pretrained LLM on a smaller, task-specific dataset.
  - The objective is to specialize the model for a particular use case or domain.
- **Why Fine Tune LLMs?**
  1. Improve performance on specific tasks.
  2. Inject domain-specific knowledge (legal, medical, financial, etc.).
  3. Adapt to company-specific language or tone.
  4. Reduce inference cost by limiting model size and scope.
- **Use Cases of BERT Fine-Tuning**
  - Customer Support Chatbots (trained on company FAQs).
  - Legal Document Analysis.
  - Scientific Paper Summarization.
  - Code Assistants for specific frameworks.
  - Sentiment classification for product reviews.

Typical LLM Fine Tuning Workflow

1. Choose a Pre-trained Model

Pick a base model from Hugging Face (e.g., distilbert-base-uncased for classification or gpt2 for text generation).

2. Prepare Your Dataset

Format your data appropriately. Common formats:

- For classification: CSV with text and label columns.
- For generation: Text file or JSON with prompt and completion.

3. Tokenize the Data

Convert text into tokens the model understands using a tokenizer.

4. Define Training Arguments

Set training parameters like learning rate, batch size, and number of epochs.

5. Train the Model

Use a trainer to fine-tune the model on your dataset.

6. Evaluate & Save

Check performance and save the model for future use.

7. Share (Optional):

- Push your fine-tuned model back to the Model Hub.
- Create a demo in Hugging Face Spaces.

Open in Colab